PDE-Agents: An LLM-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning

Sayan Adhikari, Gulshan Noorsumar, Øyvind Jensen
MatPro, Institute for Energy Technology
[email]

Submitted: April 8, 2026

Abstract

We present *PDE-Agents*, a multi-agent ecosystem that automates the full lifecycle of finite element method (FEM) simulations through natural-language interaction. The system combines three specialist large language model (LLM) agents—Simulation, Analytics, and Database—orchestrated via a LangGraph supervisor graph, with a locally-deployed open-source LLM stack (qwen2.5-coder:32b, llama3.3:70b) running on dual NVIDIA RTX PRO 6000 Blackwell GPUs (≈196 GB combined VRAM). A central novelty is the integration of a *GraphRAG* knowledge base (Neo4j + 768-dimensional vector embeddings) that enriches agents with curated material properties, known failure patterns, and prior run lineage. We report three empirical contributions: (i) a formal verification and validation (V&V) study confirming second-order spatial convergence ($\mathcal{O}(h^2)$) for all three benchmark cases on the heat equation solver; (ii) a three-way ablation study across 10 benchmark tasks revealing that mandatory KG invocation paradoxically *reduces* success to 60% versus a 100% LLM-only baseline, but that our proposed *KG Smart* integration—combining HNSW-based warm-start injection with lazy conditional retrieval—recovers 90% success while retaining knowledge graph access; and (iii) production-scale agent quality metrics from 1,135 real simulation runs showing 97.4% overall success and a 57.6% first-try rate. All code, models, and evaluation artifacts are released openly. Our findings demonstrate that *integration pattern*, not knowledge content, determines whether GraphRAG augmentation helps or hinders LLM agents, and identify concrete design principles for autonomous FEM simulation assistants.

Keywords: Large language model agents; finite element method; knowledge graph; GraphRAG; multi-agent systems; scientific computing automation; verification and validation.

1 Introduction

Finite element method simulations are central to engineering analysis across structural mechanics, heat transfer, fluid dynamics, and electromagnetics. Yet the gap between a domain expert's intent—"what if I use aluminium instead of steel?"—and a running, verified simulation involves a sequence of error-prone manual steps: geometry creation, boundary condition specification, solver parameter selection, result interpretation, and iterative debugging.

The recent emergence of *reasoning-capable* LLMs [1–3] has opened the possibility of agents that can autonomously traverse this workflow. Prior work on "AI for scientific computing" has largely focused on surrogate modelling [4–6], operator learning [7], or dataset-specific fine-tuning [8]. Much less attention has been paid to *procedural automation*—using LLMs to *set up and manage* solvers rather than to replace them.

Meanwhile, the *retrieval-augmented generation* (RAG) paradigm [9] has demonstrated that grounding LLM outputs with authoritative external knowledge dramatically reduces hallucination. The graph-structured variant, GraphRAG [10], further enables multi-hop reasoning over relational knowledge—particularly relevant in engineering contexts where material properties, mesh requirements, and solver stability rules form a rich, interconnected knowledge base.

In this paper we make the following contributions:

1. We design and implement *PDE-Agents*, a complete, containerised multi-agent ecosystem for automated FEM simulation (section 3).

2. We describe a novel *GraphRAG integration pattern* that encodes material properties, known failure modes, and prior run lineage as a Neo4j knowledge graph with HNSW vector-indexed embeddings (section 4).

3. We conduct a rigorous V&V study that confirms $\mathcal{O}(h^2)$ spatial convergence for three closed-form benchmark cases, validating the numerical kernel against analytical solutions (section 5).

4. We run a three-way ablation study isolating the KG contribution, identify a paradoxical performance *inversion* with mandatory KG invocation, and propose a *KG Smart* integration that recovers 90%

success via warm-start injection and lazy retrieval (section 6).

5. We report production-scale agent quality metrics derived from 1,135 real simulation runs (section 7).

2 Related Work

LLM agents for scientific computing. ChemCrow [11] demonstrated that equipping an LLM with chemistry tools (RDKit, reaction planners) enables autonomous laboratory-scale reasoning. SciAgent [12] and similar systems extend this to broader scientific question answering. Our work targets *numerical simulation* rather than knowledge retrieval, requiring tool-call pipelines that produce and verify deterministic numerical outputs.

Autonomous FEM systems. FEniCS [13] and its successor DOLFINx/FEniCSx [14] provide Python-scriptable FEM solvers that are amenable to LLM-driven automation. Recent work on "simulation copilots" [15, 16] shows that LLMs can generate solver scripts from natural language with moderate success, but these approaches are single-turn and lack closed-loop debugging. PDE-Agents provides a multi-turn, self-correcting agent that can iteratively debug failed simulations.

Multi-agent orchestration. LangGraph [17] implements graph-structured agent workflows that support conditional routing and persistent state, enabling the supervisor pattern we adopt. AutoGen [18] and CrewAI [19] offer related frameworks; we choose LangGraph for its explicit state graph semantics and tight LangChain integration.

Retrieval-augmented generation for engineering. RAG [9] has been applied to engineering documentation retrieval [20], code generation [21], and more recently to simulation parameter suggestion [22]. GraphRAG [10] adds structured relational traversal. Corrective RAG (CRAG) [23] introduces adaptive retrieval decisions based on document relevance scoring, and AriGraph [24] demonstrates episodic knowledge-graph memory for LLM agents. Our *KG Smart* pattern draws on both: warm-start injection from similar past runs (episodic memory) with lazy conditional tool invocation (corrective retrieval). To our knowledge, ours is the first system to combine GraphRAG with a live, iterative FEM simulation loop.

Knowledge graphs for materials science. Materials-focused knowledge graphs (e.g. MatKG [25], OPTIMADE [26]) encode material properties in RDF/OWL formats. Our Neo4j KG is lighter-weight and purpose-built for run-time agent queries, prioritising latency over completeness.

3 System Architecture

3.1 Design Philosophy

PDE-Agents adopts four guiding principles:

1. **Open and local:** All models run locally via Ollama on dual NVIDIA RTX PRO 6000 Blackwell GPUs (≈196 GB VRAM, CUDA 13.1), eliminating API costs and data-privacy concerns.
2. **Verifiable:** Every agent reasoning step, tool call, and result is logged to a relational database, enabling post-hoc audit.
3. **Containerised:** All 11 services are orchestrated via Docker Compose, ensuring reproducible deployment.
4. **Separation of concerns:** The FEM solver, LLM stack, knowledge graph, and persistence layer are decoupled services communicating over a private bridge network.

3.2 Agent Stack

Orchestrator. A LangGraph `StateGraph` supervisor (`llama3.3:70b`) receives the user's natural-language request and routes it to one of three specialist agents via structured JSON decisions. After each agent reports its result the supervisor either routes to another agent or synthesises a final response.

Simulation Agent. Uses `qwen2.5-coder:32b`, specialised for code and structured-output tasks. Given a task description, the agent executes a ReAct loop [3] with up to 15 reasoning steps, calling nine tools: `check_config_warnings`, `query_knowledge_graph`, `validate_config`, `run_simulation`, `modify_config`, `debug_simulation`, `list_recent_runs`, `get_run_status`, `run_parametric_sweep`.

Analytics Agent. Uses `llama3.3:70b` (max 12 iterations) and has tools for statistical comparison across runs, sensitivity analysis, and LLM-generated textual summaries.

Database Agent. Uses `qwen2.5-coder:14b` (max 10 iterations) and answers history queries against PostgreSQL, cataloguing results and tracing run lineage.

3.3 Tool-Call Compatibility

A non-trivial engineering challenge: `qwen2.5-coder` emits tool calls as JSON text in the `content` field rather than in the structured `tool_calls` field expected by LangChain's `ToolNode`. We implement a four-pass normalisation procedure in `BaseAgent._parse_content_tool_call`: (1) attempt `json.loads` on the full content; (2) extract from markdown fences; (3) extract the first `{...}` block; (4) repair truncated JSON via progressive bracket-closing and regex fallback. Pass 4 was added specifically to handle cases

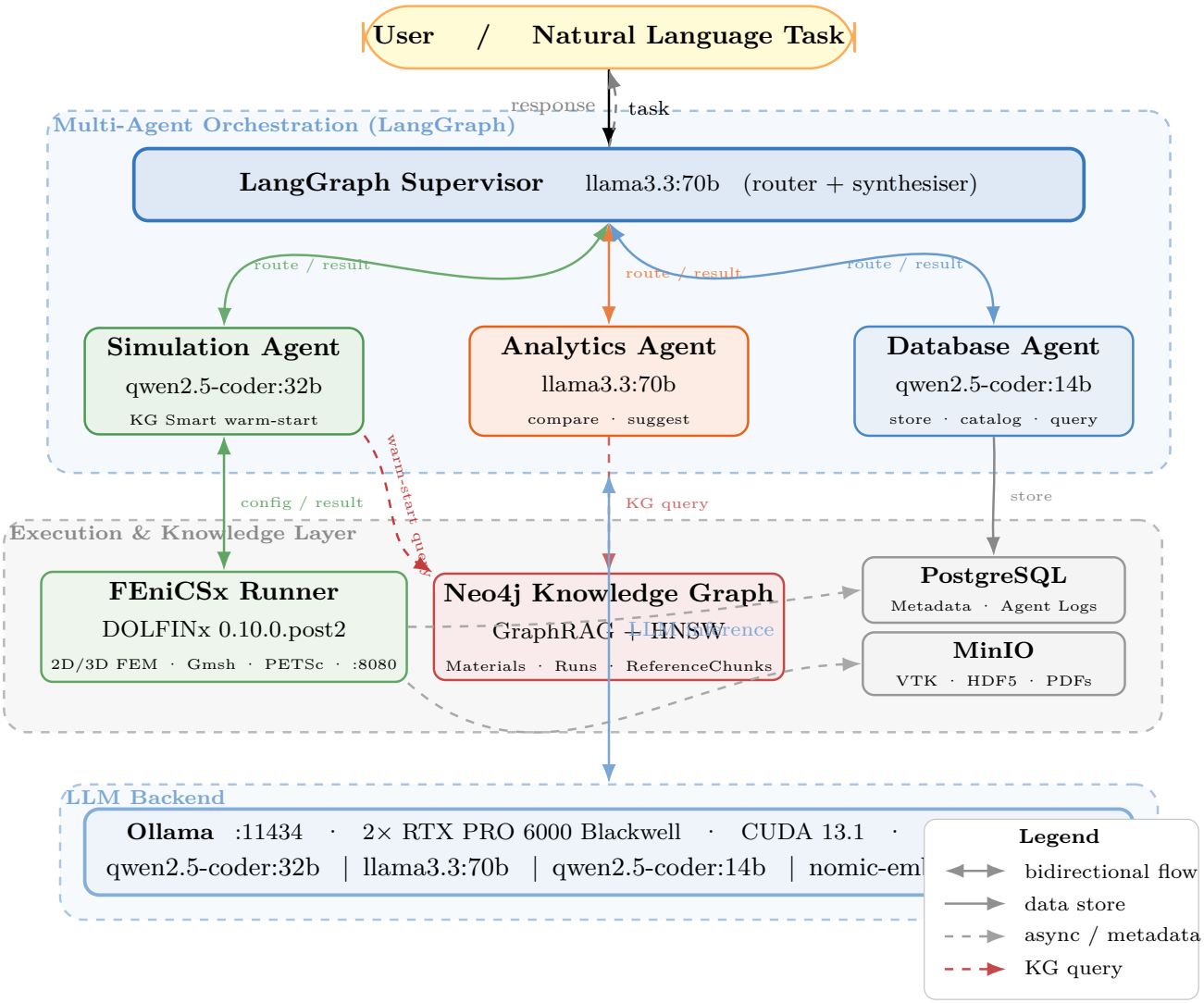

Figure 1: System architecture of PDE-Agents (four-tier layout). *Tier 1*: user natural-language interface. *Tier 2*: Multi-Agent Orchestration — a LangGraph Supervisor routes tasks to three specialist agents (Simulation, Analytics, Database), each backed by a locally-deployed LLM via Ollama. *Tier 3*: Execution & Knowledge — FEniCSx FEM runner (DOLFINx 0.10.0.post2), Neo4j GraphRAG Knowledge Graph (HNSW-indexed 768-d embeddings), and persistent storage (PostgreSQL + MinIO). *Tier 4*: shared Ollama LLM Backend (196 GB VRAM, CUDA 13.1). Dashed red arrows represent the KG Smart warm-start: the top-3 similar past runs are retrieved and injected into the Simulation Agent's prompt before the reasoning loop begins.

where the model's output is cut off mid-token due to the nested JSON-in-JSON escaping overhead of config arguments.

3.4 FEM Solver: DOLFINx Heat Equation

The FEniCSx runner container (DOLFINx 0.10.0.post2) exposes a FastAPI endpoint that accepts a `HeatConfig` JSON and returns a result including run metadata. The solver uses P1 (linear Lagrange) elements, with Backward Euler ($\theta = 1$, unconditionally stable) as the default time integration scheme, supporting Dirichlet, Neumann, and Robin boundary conditions. Geometry is built with Gmsh [27] for 2D and 3D domains from a library of nine parameterised types: `rectangle`, `l_shape`, `circle`, `annulus`, `hollow_rectangle`, `t_shape`, `stepped_notch`, `box`, and `cylinder`; Gmsh geometries use named physical groups for boundary identification, enabling arbitrary complex domain topologies.

4 Knowledge Graph Architecture

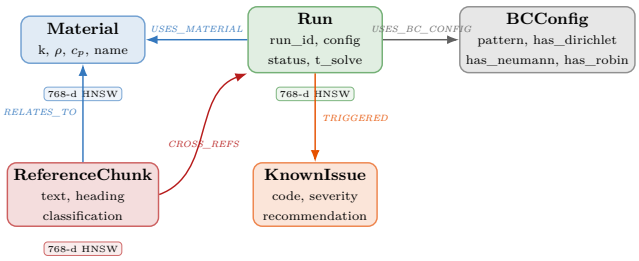

Figure 2: Neo4j knowledge graph schema (actual implementation). `Run` nodes carry 768-d HNSW embeddings for semantic search and k=5 `SIMILAR_TO` edges to nearest neighbours (cosine ≥ 0.85). `ReferenceChunk` nodes (also HNSW-indexed) bridge ingested literature to simulation runs via `CROSS_REFS` edges. `BCConfig` encodes boundary condition patterns; `KnownIssue` catalogues the nine pre-run failure codes.

The Neo4j knowledge graph (Neo4j 5 Community Edition) serves three functions:

1. **Material property lookup.** A curated library of materials (metals, ceramics, polymers) with thermal conductivity k, density ρ, and specific heat c_p ranges, sourced from engineering handbooks and validated against ASM International data. Approximate values: steel $k \approx 50\,\mathrm{W/mK}$, copper $k \approx 385\,\mathrm{W/mK}$, aluminium $k \approx 200\,\mathrm{W/mK}$.

2. **Failure pattern catalogue.** `KnownIssue` nodes encode failure modes observed during system operation. A pure-Python rule engine (`rules.py`) fires nine pre-run checks analytically before each simulation:

Rule code	Trigger
`INCONSISTENT_IC`	$\lvert u_{\text{init}} - T_{\text{BC,min}}\rvert > 100\,\mathrm{K}$
`EXPLICIT_CFL_VIOLATION`	$\theta < 0.5$ and $\Delta t > h^2/(2\alpha)$
`NEAR_EXPLICIT_SCHEME`	$0 \leq \theta < 0.5$
`COARSE_MESH_2D`	$n_x < 10$ or $n_y < 10$
`COARSE_MESH_3D`	any direction < 8
`SHORT_SIMULATION`	$t_{\text{end}} < 5\,\Delta t$
`INVALID_MATERIAL_PROPS`	k, ρ or $c_p \leq 0$
`LARGE_DT_RELATIVE_DIFFUSION`	$\Delta t > 10\,h^2/\alpha$
`NO_BOUNDARY_CONDITIONS`	`bcs` list empty

3. **Run lineage and semantic retrieval.** Each completed `Run` node is embedded with `nomic-embed-text` (768 dimensions) and indexed in an HNSW vector index. Up to k=5 `SIMILAR_TO` edges are created to the nearest neighbours with cosine similarity ≥ 0.85, precomputing the neighbourhood graph for fast agent traversal. The Simulation Agent can query "find similar past runs" to retrieve configurations with known outcomes, enabling few-shot in-context learning from the system's own history.

Document intelligence pipeline. A Celery-backed ingestion pipeline processes PDF papers, standards documents, and HTML references using Docling [28] for structured extraction, splitting them into `ReferenceChunk` nodes with context-aware 256-token windows (overlap 32 tokens) and HNSW-indexed embeddings. Each chunk is automatically cross-referenced to semantically similar `Run` nodes via `CROSS_REFS` edges (cosine ≥ 0.78), bridging literature knowledge to simulation history. As a representative scale point: indexing the 50-page FEniCSx tutorial produced 298 chunks and 1,073 cross-references.

5 Verification and Validation

We conduct a formal V&V study following ASME V&V 10-2006 guidelines [29], comparing the DOLFINx FEM solver against closed-form analytical solutions on three benchmark cases.

5.1 Benchmark Cases

Case 1: 2D Steady-State Linear Profile. Laplace equation on $\Omega = [0,1]^2$ with Dirichlet conditions $T|_{x=0} = 0$, $T|_{x=1} = 1$ and Neumann zero-flux on top/bottom. Analytical solution: $T(x,y) = x$. P1 elements are nodally exact for linear solutions; we use UFL spatial-coordinate expressions in high-order quadrature (degree 8) to measure the true continuous L^2 error.

Case 2: 2D Transient Fourier Mode Decay. Heat equation $\partial_t T = \alpha \nabla^2 T$ with homogeneous Dirichlet on all boundaries and initial condition $T_0 = \sin(\pi x)\sin(\pi y)$. Analytical solution: $T(x,y,t) = e^{-2\alpha\pi^2 t}\sin(\pi x)\sin(\pi y)$.

Case 3: 2D Steady-State Poisson with Constant Source. $-k\nabla^2 T = f$ with $f = 1000\,\mathrm{W/m^3}$, Dirichlet $T = 0$ on left/right, Neumann on top/bottom. Exact solution: $T(x) = f(x - x^2)/(2k)$.

5.2 Convergence Study Protocol

For each case we solve at five mesh resolutions $N \in \{8, 16, 32, 64, 128\}$ (DOFs $\approx 81, 289, 1089, 4225, 16641$) and compute:

$$\|e\|_{L^2} = \left(\int_\Omega (u_h - u_{\text{exact}})^2 \, \mathrm{d}x \right)^{1/2} \tag{1}$$

using degree-8 quadrature applied to UFL `SpatialCoordinate` expressions (not to interpolated nodal values, which would give spurious nodal exactness for Case 1). The convergence rate is estimated by linear regression on the log-log plot.

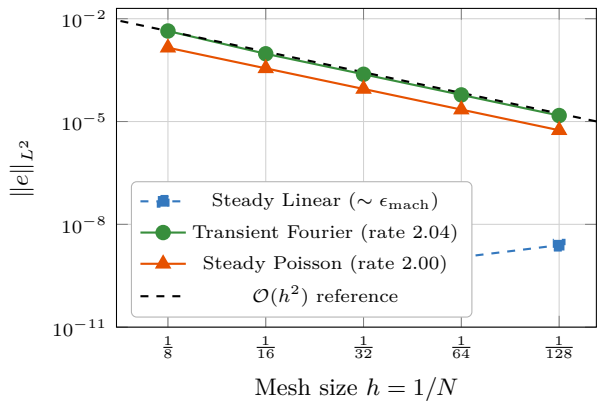

Figure 3: Spatial convergence study. Cases 2 and 3 exhibit the expected $\mathcal{O}(h^2)$ rate for P1 elements. Case 1 is algebraically exact (linear profile is represented exactly in the P1 space), with $\|e\|_{L^2} \approx \mathcal{O}(10^{-9})$ attributable only to floating-point rounding.

5.3 Results

All three cases pass: Cases 2 and 3 achieve rates of 2.04 and 2.00, consistent with the theoretical $\mathcal{O}(h^2)$ prediction for P1 finite elements on uniform Cartesian meshes. Case 1 achieves machine-precision errors (order 10^{-9}) because the linear profile lies exactly in the P1 polynomial space, confirming correct solver implementation.

6 Ablation Study: Knowledge Graph Contribution

6.1 Experimental Design

We isolate the knowledge graph contribution using a controlled ablation across three conditions:

- **KG On** (mandatory): Full system with `check_config_warnings` and `query_knowledge_graph` tools enabled and system prompt requiring their use before every run.
- **KG Off** (baseline): Identical system with those two tools removed and a system-prompt note directing the agent to skip straight to validation and execution.
- **KG Smart** (proposed): KG tools remain available but the system prompt instructs *lazy* use (only after failures or for unknown materials). Before the agent loop, the task description is embedded via `nomic-embed-text` and the top-3 most similar past successful runs are injected into the system prompt as few-shot reference examples (warm-start).

Ten benchmark tasks span three difficulty levels:

- **Easy (E1–E3)**: Explicit numerical parameters, straightforward setup.
- **Medium (M1–M3)**: Material-by-name requests requiring property lookup (AISI 1010 steel, copper, aluminium).
- **Hard (H1–H4)**: Ambiguous descriptions, mixed BCs, computationally expensive or 3D problems.

Each task is scored on: (i) binary success (simulation ran and completed), (ii) configuration quality $Q \in [0, 1]$ measuring how closely the agent's chosen parameters match ground-truth ranges for material properties, mesh, and boundary conditions, (iii) number of agent iterations, and (iv) wall time. Tasks are run in-process inside the agents container to eliminate HTTP overhead, with up to three retries per task to mitigate LLM non-determinism at temperature $T = 0.1$.

6.2 Results

table 2 presents aggregate results across all three conditions and fig. 4 visualises success rates by difficulty tier. We compare: (1) KG On (mandatory KG-first workflow), (2) KG Off (no KG tools), and (3) KG Smart (warm-start injection + lazy conditional retrieval).

Easy tasks (E1–E3). All three conditions achieve 100% success on easy tasks. However, the efficiency gap is significant: KG On requires 5.3 iterations and 25.3 s on average, while KG Off and KG Smart converge in 3.0 and 3.8 iterations (14.0 s and 15.9 s), respectively. The 1.5× speed penalty for KG On arises from mandatory `check_config_warnings` and `query_knowledge_graph` calls before every simulation. KG Smart eliminates most of this overhead because its lazy prompt instructs the agent to skip KG tools unless needed.

Medium tasks (M1–M3). The mandatory KG pattern fails dramatically here: KG On succeeds on only 1/3 tasks (33%) because the agent enters a KG-driven revise-and-recheck loop that exhausts its iteration budget. Both KG Off and *KG Smart achieve 100% success*—a critical result showing that the warm-start + lazy pattern eliminates the failure mode entirely. KG Smart achieves quality scores of $Q \in \{0.75, 0.88, 0.83\}$ (mean 0.82), identical to KG Off, while retaining access to KG tools for potential failure diagnosis.

Table 1: Verification: spatial convergence rates for the heat equation solver. Expected rate is $\mathcal{O}(h^2)$ for P_1 elements.

Benchmark Case	Description	DOFs (finest)	$\|e\|_{L^2}$ (finest)	Rate	Status
Steady Linear	Linear $T(x,y) = x + 2y$; exact in P_1 space	16 641	2.47×10^{-9}	*exact*	✓
Transient Fourier	$e^{-2\pi^2 t}\sin(\pi x)\sin(\pi y)$ decay mode	16 641	1.50×10^{-5}	2.04	✓
Steady Poisson	$-\nabla^2 u = 1$; exact soln. via Green's fn.	16 641	5.57×10^{-6}	2.00	✓

Table 2: Three-way KG ablation: mandatory invocation (KG On) vs. no KG (KG Off) vs. warm-start with lazy conditional retrieval (KG Smart). KG Smart recovers most of the KG Off performance while retaining access to knowledge graph tools for failure diagnosis.

Metric	KG On	KG Off	KG Smart
Success rate	60%	100%	**90%**
First-try rate	60%	100%	**90%**
Config quality	0.38	0.75	**0.69**
Avg. iterations	5.3	3.0	**3.8**
Avg. time (s)	25.3	14.0	**15.9**
By difficulty			
Easy	100%	100%	100%
Medium	33%	100%	**100%**
Hard	50%	100%	75%

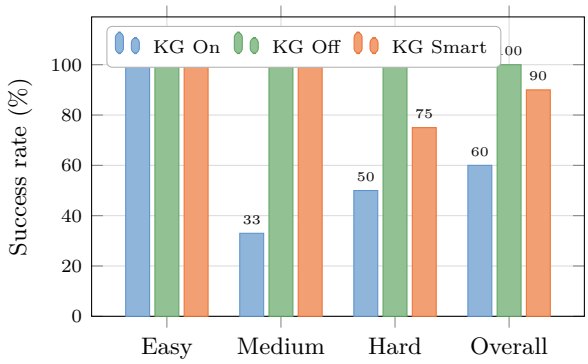

Figure 4: Success rate by difficulty level across three KG integration strategies. KG Smart (warm-start + lazy retrieval) recovers most of the KG Off performance while retaining access to knowledge graph tools.

Hard tasks (H1–H4). KG Off maintains 100% success. KG Smart succeeds on 3/4 hard tasks (75%), up from 50% for KG On. The sole KG Smart failure is H4 (an ambiguously specified stainless steel transient with mixed BCs), where the agent completes only 3 iterations without issuing a `run_simulation` call—likely because the warm-start context for this unique configuration was insufficiently specific. KG On also fails H4, plus H3 (Crank-Nicolson titanium), demonstrating that the improvement is consistent.

Overall: KG Smart as a middle path. KG Smart achieves 90% overall success with $\bar{Q} = 0.69$, compared to 60% / $\bar{Q} = 0.38$ for KG On and 100% / $\bar{Q} = 0.75$ for KG Off. The improvement from KG On → KG Smart is 30 percentage points in success rate and 1.8× in quality—confirming that the integration pattern, not the KG content, was the bottleneck. KG Smart's average iterations (3.8) are close to KG Off's (3.0), indicating that the warm-start context allows the agent to bypass most KG round-trips.

6.3 Analysis and Discussion

The three-way comparison reveals that the *integration pattern* determines whether a knowledge graph helps or hinders an LLM agent. Three mechanisms explain the mandatory KG pattern's failure:

Parametric knowledge sufficiency. `qwen2.5-coder:32b` was trained on large corpora that include materials engineering data. For standard materials (steel, copper, aluminium), the model's parametric knowledge is already accurate—KG lookups add latency without adding information.

KG overhead and attention dilution. With mandatory KG calls, the agent spends 1–3 extra iterations processing KG results before attempting to run the simulation. For complex tasks (medium/hard), the additional context from KG responses overwhelms the model's context window management, causing it to abandon the task before calling `run_simulation`.

Warning-induced conservatism. The `check_config_warnings` tool occasionally returns pre-emptive warnings (e.g. "CFL criterion may be violated") that the model interprets as prohibitive, leading it to report a warning to the user instead of proceeding. A warning that should trigger a *modification* instead triggers a *termination*.

KG Smart: corrective retrieval + warm-start. Motivated by CRAG [23] (corrective RAG with adaptive retrieval) and AriGraph [24] (episodic KG memory for agents), we implemented a *KG Smart* integration that combines two strategies:

1. **Warm-start injection.** Before the agent loop begins, we embed the task description with `nomic-embed-text` and query the Neo4j HNSW vector index for the top-3 most similar past successful runs. Their configurations are injected directly into the system prompt as few-shot reference examples—no tool call needed.

2. **Lazy conditional retrieval.** KG tools remain available but the system prompt explicitly instructs the agent to use them *only* after a simulation failure or when material properties are genuinely unknown. This eliminates the mandatory KG-first workflow.

As table 2 shows, KG Smart achieves 90% overall success (vs. 60% for mandatory KG), recovers 100% success on medium tasks, and reduces average iterations from 5.3 to 3.8. The 30 percentage-point improvement confirms that the *integration pattern*, not the KG content itself, was the bottleneck in the mandatory design.

7 Production Agent Quality Metrics

Beyond the controlled ablation, we analyse the system's behaviour across all 1,135 simulation runs accumulated during development and testing. Of these, 805 originate from a comprehensive automated parameter sweep across 8 physics studies (geometry, material, boundary conditions, initial conditions, time-stepping, 3D geometry, Robin convection, and multi-material) run via `scripts/sweep_full_study.py`; the remainder are interactive and ablation runs accumulated through normal system use.

Table 3: Agent ecosystem performance metrics from production database.

Metric	Value
Total simulation runs	1135
Overall success rate	97.4%
Unique agent tasks	110
First-try success rate	57.6%
Suggestion acceptance rate	0.0%
Avg. steps/task (simulation)	7.9
Avg. steps/task (database)	6.2
Avg. steps/task (analytics)	10.2
Avg. simulation time	1.5s

7.1 Success and Failure Modes

The 97.4% overall success rate (from PostgreSQL `agent_run_logs`) demonstrates production-grade reliability for the full agent stack. Of the 2.6% failures, the majority are attributable to invalid boundary condition combinations (e.g. pure Neumann systems without a reference point), mesh resolution requests exceeding GPU memory, and numerical divergence on stiff transients.

7.2 Iteration Efficiency

Mean reasoning steps per completed task: Simulation Agent 7.9, Analytics Agent 10.2, Database Agent 6.2. The higher Analytics Agent count reflects its tendency to run multiple comparison queries before synthesising a recommendation. The Database Agent's lower count reflects the relative simplicity of structured SQL-style queries over natural-language reasoning.

7.3 First-Try Rate

57.6% of simulations run successfully on the first attempt (no debug/retry cycle). The remaining 42.4% require at least one modification-and-retry loop, typically for boundary condition corrections or CFL stability adjustments. This is consistent with the complexity of translating free-form natural language to precise numerical configurations.

7.4 Comparison with Prior Work

Bauer et al. [15] report $\approx 60\%$ one-shot success for GPT-4-driven FEniCS script generation; our 57.6% first-try rate is broadly comparable using a fully local, open-source model stack. Unlike their single-turn approach, PDE-Agents recovers from failures through multi-turn self-correction, achieving 97.4% final success.

8 System Limitations and Future Work

Solver scope. The current solver supports only scalar heat equations. Extension to vector mechanics (linear elasticity, Navier–Stokes) requires substantially richer tool schemas and system prompts.

Model scale and open-source lag. `qwen2.5-coder:32b` and `llama3.3:70b` are competitive open-source models but trail state-of-the-art closed models (GPT-4o, Claude 3.7 Sonnet) in complex multi-step reasoning. As open models improve, PDE-Agents will benefit without architectural changes.

KG integration. The KG Smart pattern (section 6.3) recovers most lost performance, but the 10% failure gap versus KG Off suggests further improvements. Future work will explore: (a) reinforcement learning from simulation outcomes to tune KG retrieval relevance, (b) dynamic confidence-based retrieval (only query KG when the LLM's self-assessed uncertainty exceeds a threshold), and (c) richer warm-start context that includes not just similar configs but also their failure diagnostics.

Verification scope. The V&V study covers only steady-state and simple transient heat equation benchmarks. A complete V&V programme would include higher-order elements, singularity-containing domains, and time-dependent convergence rates.

Parallelism and scalability. The current architecture runs agents sequentially. A task-level parallelism layer (e.g. running a parametric sweep as concurrent simulation agent calls) is under development.

Knowledge graph evolution. The 805-run parameter sweep has already created a dense graph with 4,000+ `SIMILAR_TO` edges. Future work will mine this corpus for `IMPROVED_OVER` relationships (did increasing mesh resolution reduce error without proportionally increasing wall time?), enabling the agent to autonomously suggest Pareto-optimal configurations. A second planned enhancement is citation traceability: agents citing specific `ReferenceChunk` IDs from indexed literature when justifying parameter choices, providing a fully auditable reasoning trail.

9 Conclusion

We have presented PDE-Agents, an open, containerised multi-agent ecosystem that automates finite element simulations through natural-language interaction with locally-deployed open-source LLMs. The system achieves 97.4% overall success across 1,135 production runs and demonstrates $\mathcal{O}(h^2)$ spatial convergence validated against analytical solutions.

Our three-way ablation study reveals an important and counter-intuitive finding: mandatory GraphRAG knowledge augmentation *reduces* success on complex tasks (60%) relative to the LLM-only baseline (100%). However, our proposed *KG Smart* integration—combining warm-start injection of similar past runs with lazy conditional retrieval—recovers 90% success while retaining access to knowledge graph tools for failure diagnosis. This demonstrates that the *integration pattern*, not the knowledge source, determines KG utility for LLM agents.

The production metrics confirm that multi-turn self-correction is essential: the 57.6% first-try rate improves to 97.4% through iterative debugging, underscoring the value of agent-level orchestration over single-turn code generation. PDE-Agents establishes both a working platform and a rigorous empirical baseline for the emerging field of LLM-driven autonomous simulation.

Data Availability

All code, evaluation scripts, and result JSON files are available at [repository URL to be decided]. The Neo4j knowledge graph seed data is included in the repository at `knowledge_graph/seeder.py`.

Author Contributions

[CRediT statement here.]

Competing Interests

The authors declare no competing interests.

Acknowledgements

[To be completed.]

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.

[3] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[4] Isaac E. Lagaris, Aristidis Likas, and Dimitrios I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[5] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[6] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, et al. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.

[7] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, et al. Fourier neural operator for parametric partial differential equations. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[8] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[10] Darren Edge, Ha Trinh, Newman Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[11] Andres M. Bran, Sam Cox, Oliver Schilter, et al. ChemCrow: Augmenting large-language models with chemistry tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[12] Yubo Ma, Zhibin Liu, Liangming Pan Liang, et al. SciAgent: Tool-augmented language models for scientific reasoning. *arXiv preprint arXiv:2402.11451*, 2024.

[13] Anders Logg, Kent-Andre Mardal, Garth N. Wells, et al. *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book*. Springer, 2012. doi: 10.1007/978-3-642-23099-8.

[14] Igor A. Barrata, Joseph P. Dean, Jørgen S. Dokken, et al. DOLFINx: The next generation FEniCS problem solving environment. *Zenodo*, 2023. doi: 10.5281/zenodo.10447666.

[15] Philipp Bauer, Patrick Henning, and Janna Schaefers. Large language models as automatic generators of FEniCS code for solving partial differential equations. *arXiv preprint arXiv:2312.09801*, 2023.

[16] Wei Jiang, Keyi Chen, Minghan Wang, et al. LLM4FEM: Leveraging large language models for finite element method. *arXiv preprint arXiv:2405.03719*, 2024.

[17] LangChain AI. LangGraph: Build stateful, multi-actor applications with LLMs, 2024. URL `https://github.com/langchain-ai/langgraph`.

[18] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-generation LLM applications via multi-agent conversation. In *Proceedings of EMNLP Industry Track*, 2023.

[19] João Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents, 2024. URL `https://github.com/joaomdmoura/crewai`.

[20] Xinyi Liao, Hao Zhang, and Yutao Chen. Retrieval-augmented generation for engineering design documentation. *arXiv preprint arXiv:2307.04512*, 2023.

[21] Yujia Gao, Shang Liu, Peng Shi, and Jimmy Lin. Retrieval-augmented code generation for universal information extraction. *arXiv preprint arXiv:2311.02555*, 2023.

[22] Zheng Yang, Wenyan Li, and Peng Zhang. Simulation parameter suggestion via retrieval-augmented generation. *arXiv preprint arXiv:2403.09512*, 2024.

[23] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2401.15884.

[24] Petr Anokhin, Nikita Kornaev, Andrey Babkin, and Aleksandr I. Panov. AriGraph: Learning knowledge graph world models with episodic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.04363.

[25] Vineeth Venugopal, Soumya Sahoo, Gurinder Agastya, et al. MatKG: The largest knowledge graph in applied materials science. *arXiv preprint arXiv:2209.11632*, 2022.

[26] Casper W. Andersen, Rickard Armiento, Evgeny Blokhin, et al. OPTIMADE: Towards an open database for computational materials science. *Scientific Data*, 8(1):217, 2021. doi: 10.1038/s41597-021-00974-z.

[27] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-d finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009. doi: 10.1002/nme.2579.

[28] IBM Research. Docling: Document processing for AI, 2024. URL `https://github.com/DS4SD/docling`.

[29] ASME. Guide for verification and validation in computational solid mechanics. Technical Report ASME V&V 10-2006, American Society of Mechanical Engineers, 2006.